Storage Log AI Analyzer — User Guide

Version 0.4.0 · Suresh Kumar Darisi

Project website · GitHub · Download PDF

1. What the tool does

Analyze exported Pure Storage, Dell EMC, and HPE logs locally. The tool identifies known storage and security patterns, groups evidence into findings, and creates terminal, JSON, or Markdown reports. An optional AI mode assesses allowlisted aggregate facts.

This is a command-line tool. It does not connect directly to your arrays, collect logs continuously, repair faults, or provide a hosted upload service. Treat results as an operational aid and confirm findings against the original evidence and vendor tools.

2. Requirements

- Python 3.10 or newer; Git for the source-install workflow.
- Permission to read the exported logs and write your chosen report directory.
- Internet access for initial installation. Ordinary local analysis does not require internet access.
- Optional AI requires the AI package extra, API access, and a configured key. API usage may incur charges.

3. Install on macOS or Linux

```
git clone https://github.com/darisisuresh/storage-log-ai-analyzer.git
cd storage-log-ai-analyzer
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e .
storage-log-analyzer --version
```

Expected version output: `storage-log-analyzer 0.4.0`. Required packages `ijson>=3.6.0,<4` and `tzdata>=2026.6` are installed automatically for incremental JSON parsing and portable IANA timezone support. SQLite incident state uses the Python standard library. To reproduce this release, check out tag `v0.4.0` before installing.

4. Install on Windows PowerShell

```
git clone https://github.com/darisisuresh/storage-log-ai-analyzer.git
cd storage-log-ai-analyzer
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -e .
.\.venv\Scripts\storage-log-analyzer.exe --version
```

This workflow does not require activating PowerShell scripts. For the remaining examples, either activate the environment or replace `storage-log-analyzer` with `.\.venv\Scripts\storage-log-analyzer.exe`.

5. Try the synthetic sample first

Run these from the repository directory:

```
storage-log-analyzer vendors
storage-log-analyzer validate samples/mixed-storage.log
storage-log-analyzer summary samples/mixed-storage.log
storage-log-analyzer analyze samples/mixed-storage.log --fail-on never
```

The bundled sample has five events across three vendor families. Expected analysis starts with `Overall risk: CRITICAL` and `Events: 5 | Files: 1`. It includes drive failure, authentication failure, path failure, capacity, and replication findings. These are synthetic examples, not a benchmark of detection accuracy.

`--fail-on never` prevents a detected risk from returning exit code 2. It does not hide findings or suppress input errors.

6. Prepare your own logs

Export logs through your vendor's supported process. Keep originals read-only and work with a copy outside the source repository. Use a case directory containing text, syslog text, CSV, JSON, or JSON Lines exports.

Directory scanning includes `.log`, `.txt`, `.csv`, `.json`, and `.jsonl`, recursively. Do not scan a directory that also contains generated reports: JSON report files may otherwise be treated as new inputs. Compressed or proprietary support bundles must be extracted and converted to supported text exports first.

JSON accepts one event object, an array, or an object with an `events` array. Common fields include `timestamp` or `time`, `message` or `msg` or `description`, and `severity` or `level`.

```
{"events": [{"timestamp": "2026-01-01T00:00:00Z", "severity": "critical", "message": "HPE Alletra drive 7 failed"}]}
```

7. Validate and summarize

Replace `"/path/to/storage-logs"` with your case directory. Quote paths containing spaces.

```
storage-log-analyzer validate "/path/to/storage-logs"
storage-log-analyzer validate "/path/to/storage-logs" --strict
storage-log-analyzer summary "/path/to/storage-logs" --format json -o reports/summary.json
```

Strict validation rejects malformed/non-object JSONL with exit code 1 and a file/line error. It returns exit code 2 for unknown vendors/severities or present-but-unusable timestamps. It is a recognition check, not a guarantee that every vendor field is understood. Empty or unreadable input returns exit code 1.

To force a known vendor, use `--vendor pure`, `--vendor emc`, or `--vendor hpe`. Version 0.4.0 gives this explicit choice precedence over text detection.

8. Analyze and export reports

```
storage-log-analyzer analyze "/path/to/storage-logs"
storage-log-analyzer analyze "/path/to/storage-logs" --vendor pure --format markdown -o reports/incident.md
storage-log-analyzer analyze "/path/to/storage-logs" --format json -o reports/incident.json
storage-log-analyzer analyze "/path/to/storage-logs" --min-severity high --fail-on critical
```

`--format` accepts `text`, `json`, or `markdown`. `--output` and `-o` select a destination; missing parent directories are created. Existing output files can be overwritten, so use a new case-specific filename when preserving prior reports.

`--min-severity` filters findings before the overall report risk and failure threshold are calculated. Do not interpret a filtered report as a complete assessment of the original dataset.

9. Read and triage findings

JSON and Markdown reports include evidence and recommended actions. The terminal report emphasizes matched rules and recommended actions. The overall risk is the highest severity among included findings that are not RESOLVED. Resolved history remains visible; recovery must match device/source and resource identity. Total active-state events, unique records, repeated records and identity counts are separate fields. Reopening does not create a different device/resource identity. Untimed identical failures after recovery conservatively reopen because replay cannot be distinguished from a new failure.

- `STO-001`: drive or media failure.
- `STO-002`: path or port failure.
- `STO-003`: critically high capacity.
- `STO-004`: controller or node degradation.
- `STO-005`: replication or snapshot failure.
- `STO-006`: latency or performance degradation.
- `SEC-001`: authentication failure.
- `SEC-002`: possible ransomware or destructive activity.

Preserve the original evidence, confirm the affected component, check redundancy and recovery points, and involve the storage/security team. A rule match is not proof of root cause or compromise. No matching findings does not prove that a system is healthy.

10. Redact before sharing

```
storage-log-analyzer redact "/path/to/pure-alerts.log" -o sanitized/pure-alerts.log
```

The command refuses to overwrite the input file. It masks common IPv4 addresses, email addresses, labeled storage identifiers, and long hexadecimal IDs. Redaction is heuristic: review hostnames, usernames, paths, customer names, IPv6 addresses, and free-form messages yourself.

Local reports may retain original event evidence and file paths. Ordinary report generation does not automatically sanitize all output.

11. Optional AI assessment

From your source checkout and virtual environment:

```
python -m pip install --upgrade -e '.[ai]'
export OPENAI_API_KEY='your-key'
export STORAGE_LOG_AI_MODEL='gpt-5-mini'
storage-log-analyzer analyze "/path/to/storage-logs" --ai --format markdown -o reports/ai-assessment.md
```

In Windows PowerShell, configure variables with `$env:OPENAI_API_KEY='your-key'` and `$env:STORAGE_LOG_AI_MODEL='gpt-5-mini'`. Do not commit real keys or copy them into issues.

Model choice can also be passed with `--model`. The model must support strict structured output. The output limit is 4096 tokens by default; set `STORAGE_LOG_AI_MAX_OUTPUT_TOKENS` to an integer from 256 to 8192 to change it. Token exhaustion is reported as an AI failure.

The AI may prioritize up to eight aggregate finding groups using only supplied references and approved observation/action codes. Local validation rejects unknown evidence, contradictory states, unsupported actions, invented confidence numbers and extra claims. References identify local aggregate findings, not proof of a root cause. Confidence is explicitly uncalibrated, uncertainty is mandatory, and an operator must validate every recommendation. Use a model available to your account. The tool sends only fixed rule metadata, allowlisted counts, state, and finding

references. Raw log messages, evidence, source paths, device names, and arbitrary input fields are not sent. Review your data policy first. The request uses `store=False`; this disables response storage but does not guarantee zero provider retention. The SDK has a 60-second request timeout and up to two retries, so total elapsed time can exceed 60 seconds.

If AI fails, the local findings are retained with `ai_status=failed` and a safe error message. The command returns exit code 1. File output is checkpointed before the request and then updated with the AI outcome. Check connectivity, key configuration, model access, quota or output budget; run without `--ai` when you want only local analysis. No real API call is needed for the automated regression tests.

Preview the exact AI request without contacting a provider

```
storage-log-analyzer analyze samples/mixed-storage.log --ai-preview -o reports/ai-request.json
```

No API key or AI dependency is required for preview. `--ai-preview` cannot be combined with `--ai`. The saved JSON is the same request-body structure sent by AI mode; it contains no API key.

Incident and memory limits

Events stream through bounded counters. A temporary SQLite database stores per-device incident state and local evidence; it is deleted when the scan ends, including error paths. Ensure sufficient private temporary disk space. This database is not an encrypted evidence archive.

Reports aggregate by rule/state with up to five evidence and device samples per group. Counts remain exact. The scan accepts at most 10,000 input files; records have a 1 MiB/1,048,576-character budget, JSON nesting is limited to 64, and identity fields to 256 characters. Oversized records fail explicitly. The `parse_file` Python helper is limited to 10,000 events/8 MiB text; use `iter_file` for larger inputs.

Usable timestamps are normalized to UTC and ordered across files in temporary SQLite storage. Original text and normalization status are retained in events and bounded local evidence. Naive timestamps use `--assume-timezone UTC` by default and record that assumption; choose a different IANA zone explicitly when appropriate. RFC3164 syslog without a year requires `--assume-year`. Ambiguous/nonexistent DST wall-clock times and invalid/date-only clocks are diagnosed rather than guessed. Untimed events retain input order after timed events; untimed recovery cannot resolve a timed active failure. When identity is absent, correlation is source-file scoped.

Temporary storage is not encrypted and can grow with input size and unique identities. Cleanup occurs on normal completion and handled exceptions; abrupt process termination may leave private scratch directories. See P1 evidence and limits.

12. Exit codes and automation

For `analyze`:

- `0`: completed; no finding reaches the selected failure threshold, or `--fail-on never` was used.
- `1`: input, output, or AI assessment error.
- `2`: reported risk reaches `--fail-on` (default `high`).

For strict `validate`, exit code `2` means unknown vendors or severities. Argument errors also use the standard CLI exit code `2`; distinguish those using the command and error message.

```
storage-log-analyzer analyze "/path/to/storage-logs" --format json -o reports/latest.json --fail-on high
result=$?
case "$result" in
  0) echo "Completed below the selected risk threshold" ;;
  1) echo "Input, output, or assessment error" ;;
  2) echo "Review findings or command arguments" ;;
```

```
esac
```

13. Update and verify

```
git pull
python -m pip install --upgrade -e .
# For users of optional AI:
python -m pip install --upgrade -e '.[ai]'
storage-log-analyzer --version
```

Version 0.4.0 requires OpenAI SDK 3.42.1 or newer, below 4, for the AI extra. Use a separate environment if another application needs an older SDK. GitHub release assets are available; this project has not published a package to PyPI as part of this release.

14. Troubleshooting

- Command not found: activate the environment or run `python -m storage_log_analyzer --help` using the installed environment's Python.
- No supported events: check the path, supported extensions, readable permissions, and whether the file is empty.
- JSON must contain an object or array: correct the input structure; an `events` field must contain an array.
- Unknown vendor: use the correct `--vendor` or submit a synthetic example through the issue form.
- Exit code 2 after a report: inspect findings and the selected failure threshold; the report may have completed normally.
- Permission denied: choose a readable input and writable report directory; do not default to elevated privileges.
- AI unavailable: install `.[ai]` in the same environment, configure a key, or use local analysis without `--ai`.

15. Further help

Run `storage-log-analyzer --help` or `storage-log-analyzer analyze --help`. See the operator procedure, release notes, and changelog. Report ordinary bugs using the GitHub issue template with synthetic evidence. Report sensitive vulnerabilities privately to darisi.suresh@ieee.org as described in SECURITY.md.

16. Strict parsing, device inventory, and chronology

```
storage-log-analyzer validate /path/to/logs --strict-format
storage-log-analyzer analyze /path/to/logs --strict-format --quarantine /private/case/rejected.jsonl
storage-log-analyzer analyze /path/to/logs --inventory samples/device-inventory.json --assume-timezone
America/New_York --assume-year 2026
```

Quarantine contains original rejected records and must be outside the scanned directory. It is written with private file permissions and is not sent to AI. Lenient parsing remains available and records malformed/non-object JSONL counts in diagnostics.

The inventory format is a JSON object with a `devices` mapping, for example `{"devices":{"array-a":{"vendor":"pure"}}}`. Use canonical `pure`, `emc` or `hpe`. Precedence is forced `--vendor`, explicit structured vendor, inventory, message hints, then consistent same-device history. Another device never supplies a vendor guess; conflicting history remains unknown and inventory conflicts are diagnosed.

17. Contextual severity and replication

Supported structured fields include `redundancy` (`healthy`, `lost`, `degraded`), boolean `service_impact`, `protection_group`, `last_successful_replication` (timestamp), and nonnegative `replication_lag_seconds`.

Device identity uses `host`, `hostname`, `array`/`array_id`, or `device`/`device_id`; resource identity can use `resource`, `disk_id` or `drive_id`.

```
{"timestamp":"2026-11-02T10:00:00Z","host":"array-a","vendor":"pure","disk_id":7,"message":"disk failure detected","redundancy":"healthy","service_impact":false,"protection_group":"pg-a"}
```

A drive failure is lowered from the conservative default only with confirmed healthy redundancy and no service impact. Multiple active drives in one explicitly supplied protection group, confirmed storage service impact or redundancy loss can raise severity. The report includes `severity_basis`; these are deterministic policies, not calibrated risk scores.

```
storage-log-analyzer analyze /path/to/logs --rpo-target-seconds 300 --format json -o reports/replication.json
```

An RPO target is never assumed. Lag uses an explicit `replication_lag_seconds` value or the difference between a usable event clock and `last_successful_replication`. Successful resync resets current lag, preserves the maximum observed lag, and records current-cycle duration when clocks support it. Unknown lag stays null. These observations do not establish actual data loss or continuous maximum lag between observations.